Deutsch’s Algorithm Theory

7.2 Parity

7.2.1 The Problem

For the first algorithm, we have two unknown bits b_0 and b_1, and we want to find the parity of the two bits. That is, we want to find $b_0 \oplus b_1$, or equivalently, whether the number of 1's is even or odd. To do this, we are given an oracle $f(x) = b_x$ that takes as input an index $x \in \{0,1\}$ and returns the corresponding bit. That is,

$$f(0) = b_0, \quad f(1) = b_1.$$

We will show that to find the parity of b_0 and b_1, we must query this oracle twice classically, but only once quantumly.

7.2.2 Classical Solution

Classically, we need to know both bits in order to find $b_0 \otimes b_1$. So, we need to query the oracle twice, once to find b_0 and again to find b_1:

$$b_0 = f(0), \quad b_1 = f(1).$$

7.2.2 Classical Solution

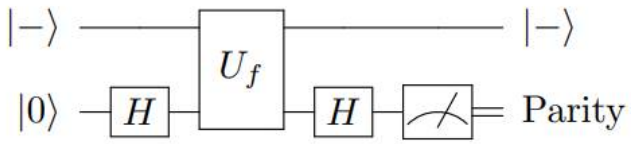
Classically, we need to know both bits in order to find $b_0 \otimes b_1$. So, we need to query the oracle twice, once to find b_0 and again to find b_1:

$$b_0 = f(0), \quad b_1 = f(1).$$

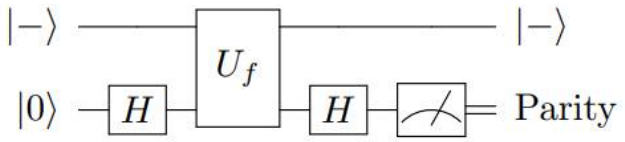
Then we take the XOR of b_0 and b_1 to find the parity. Thus, the classical query complexity is 2.

7.2.3 Quantum Solution: Deutsch's Algorithm

Quantumly, it only takes 1 query using *Deutsch's algorithm*. It uses one input qubit and one answer qubit, and the algorithm is shown in the following quantum circuit:



Let us work through each step of the circuit. Ignoring the answer qubit, we first apply the Hadamard gate to the input qubit:



Let us work through each step of the circuit. Ignoring the answer qubit, we first apply the Hadamard gate to the input qubit:

$$|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle).$$

Next, we query the oracle, which acts as a phase oracle because the answer qubit is $|-\rangle$. From Exercise 7.3, we get

$$\frac{1}{\sqrt{2}}\left[(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\right].$$

To show why this is helpful, let us rewrite this. First, we substitute $f(0) = b_0$ and $f(1) = b_1$:

$$\frac{1}{\sqrt{2}}\left[(-1)^{b_0}|0\rangle + (-1)^{b_1}|1\rangle\right].$$

Then, we factor out $(-1)^{b_0}$.

$$(-1)^{b_0}\frac{1}{\sqrt{2}}\left[|0\rangle + (-1)^{b_1-b_0}|1\rangle\right].$$

Now, depending on whether b_0 and b_1 are equal, this is

$$\begin{cases} (-1)^{b_0}\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), & b_0 = b_1 \\ (-1)^{b_0}\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), & b_0 \neq b_1 \end{cases}.$$

These are just $|+\rangle$ and $|-\rangle$, each with a phase:

$$\begin{cases} (-1)^{b_0}|+\rangle, & b_0 = b_1 \\ (-1)^{b_0}|-\rangle, & b_0 \neq b_1 \end{cases}.$$

So, if b_0 and b_1 are equal, we have $|+\rangle$ with an overall phase, and if b_0 and b_1 are unequal, we have $|-\rangle$ with an overall phase. We can distinguish these by measuring in the X-basis $\{|+\rangle, |-\rangle\}$. Or, we can apply the Hadamard gate again and then measure in the Z-basis $\{|0\rangle, |1\rangle\}$. Applying the Hadamard gate, we get

$$\begin{cases} (-1)^{b_0}|0\rangle, & b_0 = b_1 \\ (-1)^{b_0}|1\rangle, & b_0 \neq b_1 \end{cases}.$$

If we measure this, we either get $|0\rangle$ or $|1\rangle$, since the overall phase of $(-1)^{b_0}$ does not matter. If we get $|0\rangle$, we know that $b_0 = b_1$, and so the parity of the bits is 0 (even). On the other hand, if we get $|1\rangle$, we know that $b_0 \neq b_1$, and so the parity of the bits is 1 (odd). Thus, depending on whether we get $|0\rangle$ or $|1\rangle$, we know whether the parity of the two bits is 0 or 1, and we determined this with just one query to the oracle. Thus, the quantum query complexity is 1, which is an improvement over the classical query complexity of 2. While the improvement in query complexity from 2 to 1 may be small, it is our first concrete example of a quantum computer outperforming a classical computer.

Note in Deutsch's algorithm, we never learned the values of b_0 and b_1 themselves, which would require two oracle queries. Instead, we only learned whether they are equal or opposite, which corresponds to even or odd parity, respectively.

Here's the clean intuition—no clutter.

🔹 What's the goal?

You want the **parity**:

$$b_0 \oplus b_1$$

- 0 → even (same bits)
- 1 → odd (different bits)

🔹 Classical vs Quantum

- **Classical:** must query twice → get b_0, b_1
- **Quantum (Deutsch):** 1 query only

🔹 The Trick (Core Idea)

You **don't learn the bits**.
You only learn: *are they same or different?*

🔹 Step-by-step intuition

1. Start

Input qubit:

$$|0\rangle \to H \to \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

---

2. Oracle (Phase Kickback ⚡)

Because second qubit is $|-\rangle$, the oracle encodes **values as phases**:

$$\frac{1}{\sqrt{2}}\left[(-1)^{b_0}|0\rangle + (-1)^{b_1}|1\rangle\right]$$

👉 This is the key:

- No values revealed

- Only **phase difference** stored

---

3. Factor phase

$$(-1)^{b_0} \cdot \frac{1}{\sqrt{2}} \left[|0\rangle + (-1)^{b_1 - b_0}|1\rangle\right]$$

Now only this matters:

- $b_0 = b_1 \to$ phase = + $\to |+\rangle$
- $b_0 \neq b_1 \to$ phase = − $\to |-\rangle$

---

4. Final Hadamard

$$|+\rangle \to |0\rangle, \quad |-\rangle \to |1\rangle$$

🔹 Final Answer (what you measure)

- Measure **0** $\to b_0 = b_1 \to$ parity = 0
- Measure **1** $\to b_0 \neq b_1 \to$ parity = 1

🔥 Big Insight

Quantum advantage comes from:

> **Encoding function values into phase, then interfering them**

Instead of:

- "What are the values?"
  You ask:
- "Do they cancel or reinforce?"

🔸 One-line summary

Deutsch's algorithm uses **phase kickback + interference** to convert

$$(b_0, b_1) \;\to\; (b_0 \oplus b_1)$$

in **one query**.

If you want, I can walk you through **phase kickback visually**—that's the part that usually clicks everything.

OpenAI

7.2.4 Generalization to Additional Bits

What if we have n bits $b_0, b_1, \ldots, b_{n-1}$, and we want to find their parity? Classically, we need to know all n bits, so it takes n queries. Quantumly, we can use Deutsch's algorithm to find the parity of pairs of bits:

$$\underbrace{b_0, b_1}_{\text{parity}}, \underbrace{b_2, b_3}_{\text{parity}}, \ldots \underbrace{b_{n-2}, b_{n-1}}_{\text{parity}}.$$

This takes $n/2$ queries. Then we can take the XOR of all these parities to get the parity of all the bits. This takes no additional queries. So, the quantum query complexity is $n/2$, which is half classical query complexity. Note both the classical and quantum runtimes are $O(n)$, however, so there is no improvement in their asymptotic scaling.

7.3 Constant vs Balanced Functions

7.3.1 The Problem

In this problem, we have a function $f(x)$ that takes as input a binary number $x = x_{n-1} \ldots x_1 x_0$ of length n and outputs 0 or 1. Mathematically, we can write this as $f : \{0,1\}^n \to \{0,1\}$, where $\{0,1\}^n$ denotes bit strings of length n. We additionally have the promise that f is constant (always outputs 0 or always outputs 1) or *balanced* (outputs 0 half the time, and outputs 1 half the time), and the problem is to determine which we have. Put another way, f outputs 1 none of the time, all of the time, or half of the time, and the task is to determine if it is none or all of the time (constant) or half of the time (balanced).

For example, the following function on binary strings of length 3 (i.e., on 3 bits) is balanced since it outputs 0 half the time and 1 the other half of the time:

x_2	x_1	x_0	$f(x)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Note $n = 1$ is Deutsch's algorithm since the input is a single bit, and if the function is constant, the parity is 0 or even, and if the function is balanced, the parity is 1 or odd. So the Deutsch-Jozsa algorithm can be seen as a generalization of Deutsch's algorithm.

7.3.2 Classical Solution

Classically, to determine with certainty whether f is constant or balanced, we need to query half the inputs, plus one, in the worst case scenario. That is, if we query half the inputs and get zero each time, then we still do not know if just half the outputs are zero, or if all the outputs are zero. Querying one more input resolves this. Since there are 2^n possible inputs (binary strings of length n), the classical query complexity is $2^{n-1} + 1$. Note this scales exponentially in n, i.e., it is $O(2^n)$.

In practice, however, one may accept a classical algorithm that guesses the correct answer most of the time. For example, say we query f with $c = 10$ different random inputs, and we get $f = 0$ each time. Then, we can guess that f is constant with some degree of certainty. As we will show next, the probability that our guess is wrong can be made smaller than any constant using some suitable constant value for c. Then, such a randomized algorithm can solve the problem with just c queries, which is $O(1)$.

To show this, say we classically query f for c different random inputs. If we get the same output every time, we guess that f is constant, and if we get a mix of 0's and 1's, we guess that f is balanced. Let us calculate the probability that these guesses are incorrect. First, if f is constant, we will get the same output for all c of our inputs, and we will correctly guess that f is constant, so there is no error in this case. If f is balanced and the c outputs are any mix of 0's and 1's, we will correctly guess that f is balanced, so there is no error in this case, either. If f is

To show this, say we classically query f for c different random inputs. If we get the same output every time, we guess that f is constant, and if we get a mix of 0's and 1's, we guess that f is balanced. Let us calculate the probability that these guesses are incorrect. First, if f is constant, we will get the same output for all c of our inputs, and we will correctly guess that f is constant, so there is no error in this case. If f is balanced and the c outputs are any mix of 0's and 1's, we will correctly guess that f is balanced, so there is no error in this case, either. If f is balanced and all c of our outputs are the same, however, we will incorrectly guess that f is constant, which is an error. Let us find the probability of this error. Say all c of our outputs are 0, but f is actually balanced. To get the first 0, there are $2^n/2 = 2^{n-1}$ outputs that are 0 out of a total of 2^n outputs, so the probability of getting a 0 is $2^{n-1}/2^n$. For the second 0, there are $2^{n-1}-1$ outputs remaining that are 0 out of a total of 2^n-1 outputs remaining, so the probability of getting a second 0 is $(2^{n-1}-1)/(2^n-1)$. Continuing this reasoning, the probability of getting c zeros is

$$\begin{aligned}
&\frac{2^{n-1}}{2^n}\frac{2^{n-1}-1}{2^n-1}\frac{2^{n-1}-2}{2^n-2}\cdots\frac{2^{n-1}-(c-1)}{2^n-(c-1)} \\
&\approx \frac{2^{n-1}}{2^n}\frac{2^{n-1}}{2^n}\frac{2^{n-1}}{2^n}\cdots\frac{2^{n-1}}{2^n} \\
&= \frac{1}{2}\frac{1}{2}\frac{1}{2}\cdots\frac{1}{2} = \frac{1}{2^c},
\end{aligned}$$

where in the second line, we have approximated the expression for large n. Similarly, the probability of all c queries yielding 1 even though f is balanced is also $1/2^c$. Together, the total probability of incorrectly guessing that f is constant when it is actually balanced is

$$\frac{1}{2^c}+\frac{1}{2^c}=\frac{1}{2^{c-1}}.$$

This does not depend on n. So, we can bound the error by checking an appropriate number of inputs. For example, if we want the probability of error to be less than 1%, we only need to query f for $c = 8$ different inputs, and this is a constant number of queries regardless of n. Thus, this randomized algorithm takes $O(1)$ queries of f.

From Exercise 1.53, problems that are efficiently solved with bounded error by such randomized algorithms are contained in the complexity class *bounded-error probabilistic polynomial time* (BPP). It is believed that $P = BPP$, but it is not proven. Since we gave an efficient randomized algorithm for determining whether f is constant or balanced, this problem is in BPP.

To review, for a classical computer to determine with certainty whether f is constant or balanced, it needs $2^{n-1} + 1$ queries to f in the worst case, which is $O(2^n)$. For a probabilistic classical computer to guess the answer with bounded error, it only needs a constant number of queries to f, which is $O(1)$.

Here's the distilled story of both sections—just the essence.

🔹 7.2.4 Generalization (Parity for n bits)

Goal

Compute:

$$b_0 \oplus b_1 \oplus \cdots \oplus b_{n-1}$$

Classical

- Need **all bits**
- Queries = **n**

Quantum (using Deutsch repeatedly)

- Compute parity of **pairs**:

$$(b_0 \oplus b_1), (b_2 \oplus b_3), \ldots$$

- Each pair → **1 query**
- Total queries = **n/2**

Then XOR all results (free, no oracle).

Final result

- **Quantum queries = n/2**
- **Classical queries = n**

👉 Only a **factor-2 speedup**, not exponential
👉 Still $O(n)$, so no asymptotic advantage

🔹 7.3 Constant vs Balanced (Deutsch–Jozsa)

Problem

Function:

$$f : \{0,1\}^n \to \{0,1\}$$

Promise:

- **Constant** → same output always
- **Balanced** → 0 half, 1 half

Goal: decide which.

🔹 Classical Complexity

Deterministic

- Worst case:

$$2^{n-1} + 1$$

👉 Exponential $O(2^n)$

Randomized (important insight)

- Query **c random inputs**
- If all outputs same → guess constant
- If mixed → balanced

Error probability:

$$\frac{1}{2^{c-1}}$$

👉 So:

- Constant queries (like $c = 8$) → very small error
- Complexity = **O(1)**

🔹 Quantum (Deutsch–Jozsa)

- Uses **superposition + phase kickback + interference**
- Evaluates **all inputs at once**

👉 Result:

- **1 query**
- **Exact answer (no error)**

🔥 Big Picture

Approach	Queries	Accuracy
Classical deterministic	$O(2^n)$	Exact
Classical randomized	$O(1)$	Small error
Quantum (DJ)	**1**	**Exact**

🔹 Deep Insight

- Deutsch (n=1): finds **parity of 2 values**
- Deutsch–Jozsa: finds **global property of function**

👉 Same trick:

> Encode answers in **phase**, then extract via **interference**

🔸 One-line intuition

Quantum computes a **global property** without learning individual values.

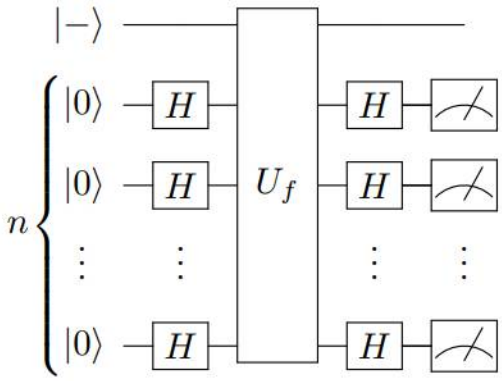
If you want, next step is the **actual Deutsch–Jozsa circuit walkthrough**—that's where everything clicks visually.

7.3.3 Quantum Solution: Deutsch-Jozsa Algorithm

A quantum computer using the *Deutsch-Jozsa algorithm* can determine with certainty whether f is constant or balanced using just 1 query to f. This is an exponential speedup over the exact classical algorithm, but no speedup over the bounded-error probabilistic classical algorithm.

The Deutsch-Jozsa algorithm is very similar to Deutsch's algorithm, but we now have n qubits (plus an answer qubit, which we ignore by using a phase oracle). These n qubits are initially each in the $|0\rangle$ state, and we apply Hadamards to put them in a superposition of all n-bit strings. Then, we query the oracle on this superposition.

Finally, we apply Hadamards to all the qubits to create a state that we measure, and whose measurement outcome allows us to distinguish whether the function is constant or balanced. Including the answer qubit, the Deutsch-Jozsa algorithm as a quantum circuit is



Let us work out the math to show why this determines whether f is constant or balanced. Ignoring the answer qubit, we begin with n qubits, all in the $|0\rangle$ state. First applying the Hadamard gate to each of these n qubits, we get

$$
\begin{aligned}
|0\rangle^{\otimes n} &\xrightarrow{H^{\otimes n}} |+\rangle^{\otimes n} \\
&= \frac{1}{\sqrt{2^n}} (|0\rangle + |1\rangle)^{\otimes n} \\
&= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle. \qquad (7.1)
\end{aligned}
$$

So, applying Hadamards to the all zero state creates a uniform superposition over all binary strings. Next, we query the phase oracle:

$$
\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \xrightarrow{U_f} \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle.
$$

Finally, we again apply the Hadamard gate to each of the n qubits:

$$
\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle \xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle. \qquad (7.2)
$$

This is the final state of the algorithm before the measurement, and to interpret this, let us focus on just one $H^{\otimes n}|x\rangle$, where $|x\rangle$ is a single n-bit string:

$$
H^{\otimes n}|x\rangle = H^{\otimes n}|x_{n-1} \ldots x_1 x_0\rangle
$$

This is the final state of the algorithm before the measurement, and to interpret this, let us focus on just one $H^{\otimes n}|x\rangle$, where $|x\rangle$ is a single n-bit string:

$$
H^{\otimes n}|x\rangle = H^{\otimes n}|x_{n-1} \ldots x_1 x_0\rangle
$$

$$
= H|x_{n-1}\rangle \ldots H|x_1\rangle H|x_0\rangle.
$$

Depending on whether x_i is 0 or 1, $H|x_i\rangle$ is either $|+\rangle$ or $|-\rangle$. To account for the difference in sign between $|+\rangle$ and $|-\rangle$, we can write $H|x_i\rangle$ as

$$
H|x_i\rangle = \frac{1}{\sqrt{2}} \Big[|0\rangle + (-1)^{x_i} |1\rangle \Big].
$$

This way, when $x_i = 0$, $(-1)^{x_i} = 1$, and the result is $|+\rangle$, and when $x_i = 1$, $(-1)^{x_i} = -1$, and the result is $|-\rangle$. Writing each $H|x_i\rangle$ like this, we get

$$\begin{aligned} &H|x_0\rangle H|x_1\rangle \dots H|x_{n-1}\rangle \\ &= \frac{1}{\sqrt{2}}\Big[|0\rangle + (-1)^{x_{n-1}}|1\rangle\Big] \dots \frac{1}{\sqrt{2}}\Big[|0\rangle + (-1)^{x_1}|1\rangle\Big]\frac{1}{\sqrt{2}}\Big[|0\rangle + (-1)^{x_0}|1\rangle\Big]. \end{aligned}$$

Multiplying out the terms, this becomes

$$\begin{aligned} \frac{1}{\sqrt{2^n}}\Big[&|0\dots000\rangle + (-1)^{x_0}|0\dots001\rangle + (-1)^{x_1}|0\dots010\rangle \\ &+ (-1)^{x_1}(-1)^{x_0}|0\dots011\rangle + (-1)^{x_2}|0\dots100\rangle + (-1)^{x_2}(-1)^{x_0}|0\dots101\rangle \\ &+ (-1)^{x_2}(-1)^{x_1}|0\dots110\rangle + (-1)^{x_2}(-1)^{x_1}(-1)^{x_0}|0\dots111\rangle + \dots\Big]. \end{aligned}$$

Writing $(-1)^{x_1}(-1)^{x_0}$ as $(-1)^{x_1+x_0}$, and similarly elsewhere, we get

$$\begin{aligned} \frac{1}{\sqrt{2^n}}\Big[&|0\dots000\rangle + (-1)^{x_0}|0\dots001\rangle + (-1)^{x_1}|0\dots010\rangle \\ &+ (-1)^{x_1+x_0}|0\dots011\rangle + (-1)^{x_2}|0\dots100\rangle + (-1)^{x_2+x_0}|0\dots101\rangle \\ &+ (-1)^{x_2+x_1}|0\dots110\rangle + (-1)^{x_2+x_1+x_0}|0\dots111\rangle + \dots\Big]. \end{aligned}$$

This is a sum over all n-bit strings $|z\rangle = |z_{n-1}\dots z_1 z_0\rangle$, so it becomes

$$\frac{1}{\sqrt{2^n}}\sum_{z\in\{0,1\}^n}(-1)^{\sum_{i:z_i=1}x_i}|z\rangle.$$

For the negative sign, the power is the sum of the values of x_i such that $z_i = 1$. We can also write this sum using the *dot product* $x \cdot z$,

$$x \cdot z = x_{n-1}z_{n-1} + \dots + x_1 z_1 + x_0 z_0.$$

In this dot product, the only x_i's that survive are those where $z_i = 1$. Using this notation, we get

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}}\sum_{z\in\{0,1\}^n}(-1)^{x\cdot z}|z\rangle. \tag{7.3}$$

OpenAI

Plugging this into Eq. (7.2), the final state of the algorithm before measurement is

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \frac{1}{\sqrt{2^n}} \sum_{z \in \{0,1\}^n} (-1)^{x \cdot z} |z\rangle$$
$$= \sum_{z \in \{0,1\}^n} \left(\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)+x \cdot z} \right) |z\rangle. \qquad (7.4)$$

To see how measuring this state lets us determine whether the function is constant or balanced, let us calculate the probability of getting all zeros $|0\ldots00\rangle$. The amplitude of $|0\ldots00\rangle$ (right before measurement) is

$$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)}.$$

This amplitude depends on whether $f(x)$ is constant or balanced:

- If $f(x)$ is constant, then $f(x)$ always outputs the same value, so $f(x) = f(0\ldots00)$ for all x, and the amplitude is

  $$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(0\ldots00)} = \frac{1}{2^n} (-1)^{f(0\ldots00)} 2^n = (-1)^{f(0\ldots00)}.$$

  Taking the norm-square of this, if $f(x)$ is constant, the probability of measuring $|0\ldots00\rangle$ is 1.
- If $f(x)$ is balanced, then $(-1)^{f(x)}$ is 1 half the time and -1 the other half the time, so the amplitude is 0. Hence, if $f(x)$ is balanced, the probability of measuring $|0\ldots00\rangle$ is 0, so we are guaranteed to get something other than $|0\ldots00\rangle$ when we measure.

Thus, to determine if f is constant or balanced, we measure the n qubits, and if we get $|0\ldots00\rangle$, the function is constant, and if we get anything else, the function is balanced.

Here's the **Deutsch–Jozsa algorithm** distilled to its core logic—what all that math is really doing.

🔹 Big Idea

You evaluate **all 2^n inputs at once**, encode answers as **phases**, then use **interference** to detect a global pattern.

🔹 Step-by-step intuition

1. Create superposition

$$|0\rangle^{\otimes n} \xrightarrow{H^{\otimes n}} \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$$

👉 Now you have **every input** x simultaneously.

---

2. Oracle (phase encoding ⚡)

$$|x\rangle \to (-1)^{f(x)} |x\rangle$$

👉 No output stored—only **phase flips**:

- $f(x) = 0 \to +$
- $f(x) = 1 \to -$

---

3. Interference (Hadamard again)

This mixes all amplitudes:

$$\sum_x (-1)^{f(x)} |x\rangle \xrightarrow{H^{\otimes n}} \sum_z \left(\sum_x (-1)^{f(x)+x\cdot z} \right) |z\rangle$$

👉 This scary sum is just:

Constructive vs destructive interference

🔹 The only state that matters: $|0\ldots0\rangle$

Amplitude:

$$\frac{1}{2^n} \sum_x (-1)^{f(x)}$$

🔹 Two cases

✅ If f is CONSTANT

- All terms same sign
- Sum = $\pm 2^n$

👉 Amplitude = ±1
👉 Probability = **1**

✔️ You ALWAYS measure:

$$|0\ldots0\rangle$$

❌ If f is BALANCED

- Half +1, half −1
- Perfect cancellation

👉 Amplitude = 0
👉 Probability = **0**

✔️ You NEVER get:

$$|0\ldots0\rangle$$

🔹 Final Decision Rule

- Measure **all zeros** → **CONSTANT**
- Measure **anything else** → **BALANCED**

🔥 Why this works

Quantum does:

> Turn "pattern of outputs" → "interference pattern"

Instead of checking values one by one, it checks:

- Do phases **add up** (constant)?
- Or **cancel out** (balanced)?

🔹 One-line intuition

Deutsch–Jozsa converts:

$$\text{global property of } f \;\rightarrow\; \text{single measurement outcome}$$

🔥 Deep insight (very important)

- You never learn **any individual** $f(x)$
- Yet you learn a **global truth about all of them**

If you want next: I can compress this into a **3-line exam answer** or give a **visual circuit intuition** that makes the interference step instantly obvious.